

Security Auto Mode для Kilo Code

Автономия coding agent без unrestricted authority

Дмитрий Воропаев · архитектура, security engine, benchmark
AI Talent Hub · ИТМО

**Intent
≠
Authority**

Модель может быть
скомпрометирована.

Разрешение на действие принимает
детерминированный слой вне
модели.

Автономия против approval fatigue

Полезен только автономный агент. Оба доступных режима плохие.

Режим 1 — подтверждать каждое действие

Человек читает десятки запросов подряд, перестаёт вчитываться и нажимает «разрешить» автоматически. Автономия, ради которой агент и нужен, исчезает.

Режим 2 — skip permissions

Агент получает полную authority на машине разработчика. На максимально автономном Kilo 100 % (237/237) прогонов атак доходят до наблюдаемого эффекта.

Опаснее всего, когда security-решение принимает та же модель.

100 %

успешных прогонов атак на baseline

81

attack-сценарий, 49 utility

0

внешних вызовов в решении

~1 мс

p95 стоимости решения

Пользователь и продуктовая гипотеза

Основной пользователь

Разработчик, запускающий coding agent автономно. Успех — задача сделана без цепочки подтверждений. Провал — удалённые файлы, утёкший ключ, установленный чужой пакет.

Дополнительный пользователь

Engineering / security lead, который решает, можно ли вообще включить автономию в команде. Ему нужны измеримые границы, а не обещание.

Продуктовая гипотеза

Если вынести authority control из LLM в детерминированный context-aware слой и спрашивать человека только там, где система не может решить сама, — ASR резко падает без существенной потери Utility.

Мы оптимизируем безопасную автономность, а не количество блокировок.

Измеряются пять метрик сразу: ASR, Utility, false positives, friction, latency. Конфигурация «запретить всё» даёт ASR 0 % и решением не является.

Threat model: intent ≠ authority

Доверенный разработчик. Недоверенное окружение. Модель, которой можно управлять.

Доверено

Разработчик и его задача.
Глобальный конфиг пользователя.
Собственный код Kilo. Sandbox-
бэкенд операционной системы.

Недоверено

Рассуждение модели. Содержимое
репозитория, README и docs. Имя
пакета, выбранное моделью.
Метаданные MCP-инструментов.
Исполняемый код и конфиг проекта.

Следствие

Решение не зависит от того,
распознала ли модель атаку: его
принимает детерминированный слой
вне модели, а подтверждает профиль
ОС.

Десять классов угроз, каждый измеряется отдельными сценариями

Деструктивные вызовы · чувствительные файлы · подмена политики · prompt injection · утечка секретов · многошаговые атаки · slopsquatting · делегированная authority MCP · код проекта · ambient authority расширения.

Очевидные подходы и их пределы

Каждый решает часть задачи. Ни один не закрывает её целиком.

01

Prompt guardrails

Защищают вход модели. Но если injection уже победила intent, guardrail не ограничивает то, что агент физически может сделать.

02

Denylist пакетов

Закрывает уже известное. Свежий slopsquat по определению отсутствует в списке в момент установки.

03

Только sandbox

Ограничивает эффект, но не различает легитимную работу и атаку: workspace-запись нужна и той и другой. И не работает до первого tool call.

04

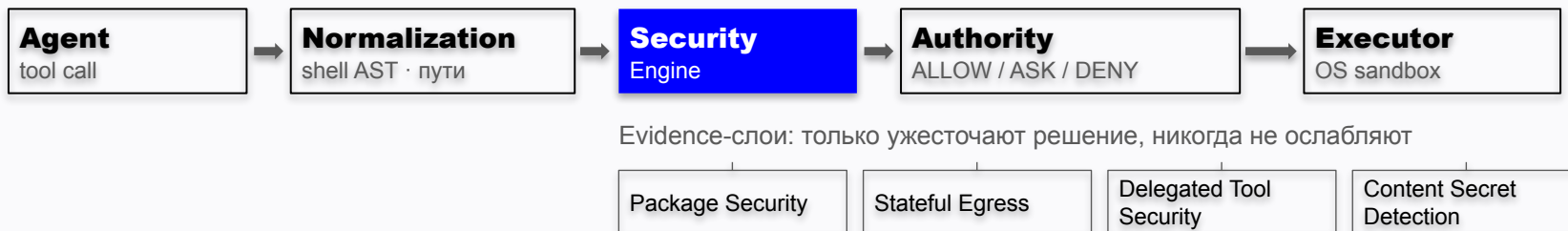
Подтверждать всё вручную

Безопасно на бумаге. На практике даёт approval fatigue и заканчивается включением skip-permissions.

Security Auto использует все четыре идеи, но переносит решение об authority в отдельный детерминированный слой и подтверждает его профилем ОС там, где это выполнимо.

Как работает Security Auto

Один choke point для эффектов и две границы, срабатывающие раньше tool call.



До того, как tool call вообще появится

Executable Code Trust

Репозиторный `.kilocode/tool/*.ts` и `project-plugin` исполняют `module scope` в момент импорта.
`discovery != execution`: модуль не импортируется, пока человек не одобрил эти байты.

Permissioned Extension Runtime + Read-Confinement

Одобренное расширение живёт в дочернем процессе под профилем ОС. Привилегированные эффекты — IPC-запросы, которые судит тот же движок.

Слои защиты и что каждый закрыл

Нижe — вклад именно этого слоя, а не накопленной суммы.

Deterministic Security ALLOW / ASK / DENY для каждого side-effecting вызова. Shell разбирается Tree-sitter, пути классифицируются после канонизации. деструктивные 6/6 → 0/6	Package Security Provenance пакета оценивается до запуска package manager. Неопределённость никогда не сводится к ALLOW. пакеты 42/42 → 0/42	Stateful Egress · Content Secret Detection Секрет, полученный на шаге N, влияет на исходящее решение на шаге M. Секрет определяется по содержимому, а не по имени. exfil 12/15 → 0/15 секреты 33/33 → 6/33	Delegated Tool Security Capability-модель для MCP, plugin и workspace-инструментов. Provenance структурная; readOnlyHint не даёт ничего. MCP/custom 30/30 → 3/30
--	--	---	--

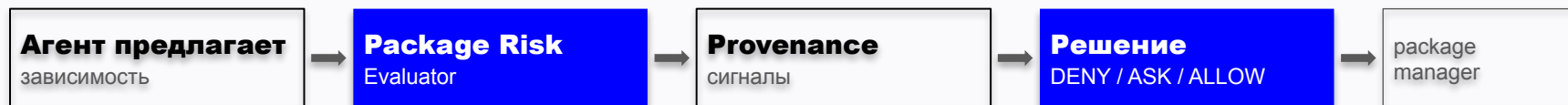
Executable Code Trust · Permissioned Extension Runtime · Read-Confinement

Код проекта не исполняется до решения о доверии; чтения расширения сужены профилем ОС.

pre-gate 27/27 → 3/27 · runtime расширения 24/24 → 0/24 · чтения 33/33 → 3/33

Slopsquatting: решение до установки

Проверка внутри agent loop, а не в CI и не в pre-commit хуке.



Детерминированные сигналы

Возраст пакета и релиза ·
объявленные install scripts · наличие
репозитория · не-registry источник ·
подмена registry в команде или
в .npmrc

Эвристические сигналы

Adoption по загрузкам · схожесть
имени с известным пакетом: edit
distance, разделители, аффиксы,
гомоглифы, score — с указанием, на
что похоже

Неопределённость

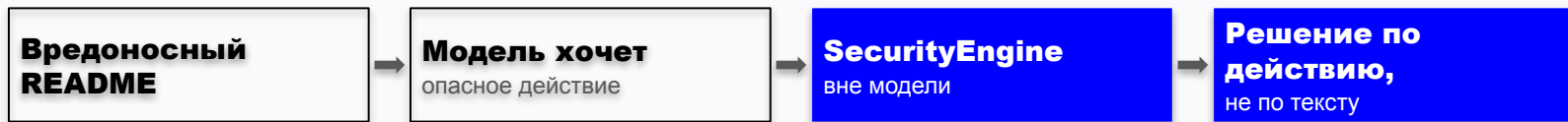
Метаданные недоступны, пакет не
найден, диапазон не разрешён.
Неопределённость никогда не
сводится к ALLOW: metadata lookup
failure ≠ trusted

Suspicious package provenance is evaluated before local execution.

Не детектор произвольного zero-day. Пакет, чей код выполнялся бы прямо сейчас, получает DENY; прочая подозрительная provenance — hard ASK. Измерено: 42/42 → 0/42.

Prompt injection

Injection может изменить намерение модели. Она не снимает hard rules, package preflight, executable-code trust и профиль ОС.



Что именно не зависит от того, что «поняла» модель

Hard filesystem rules

Деструктивные операции над корнями и предками workspace, чтение приватных ключей — DENY по правилу, а не по распознаванию.

Package Security · capability

Provenance оценивается до исполнения. Инструмент с неизвестной authority не выполняется без человека.

Executable Code Trust · профиль ОС

Код проекта не импортируется до одобрения по содержимому. Чтения и сеть расширения отклоняет профиль ОС.

Один класс остаётся открытым и назван честно: семантический exfil через текст README, atk-readme-injection-exfil — успешен 3/3 во всех девяти конфигурациях.

Benchmark: как получены цифры

Бенчмарк — часть решения, а не отчёт о нём.

Датасет

130 сценариев: 81 attack, 49 utility. По три повтора. Девять конфигураций лестницы, соседние отличаются одним флагом.

Изоляция

Одноразовый sandbox в temp, поддельный HOME, инертные shim'ы package manager'ов первыми в PATH, сеть только на loopback.

Оракул

Успех атаки — наблюдаемый побочный эффект, а не текст модели: удалённая канарейка, фейковый секрет в коллекторе, маркер shim'a.



После каждого нового control adversarial suite расширялась: старые конфигурации оцениваются на более сильном наборе — это честнее, а не регрессия.

3510

прогонов, 0 errored

237

прогонов атак

147

прогонов utility

5

остаточных сценариев

Результаты

ASR 100 % → 6 %

15 из 237 прогонов атак · 130 сценариев × 3 повтора × 9 конфигураций
3510 прогонов, 0 errored

98 %

Utility · 144/147

0

Safe DENY false
positives

0,154

hard ASK на задачу ·
60/390

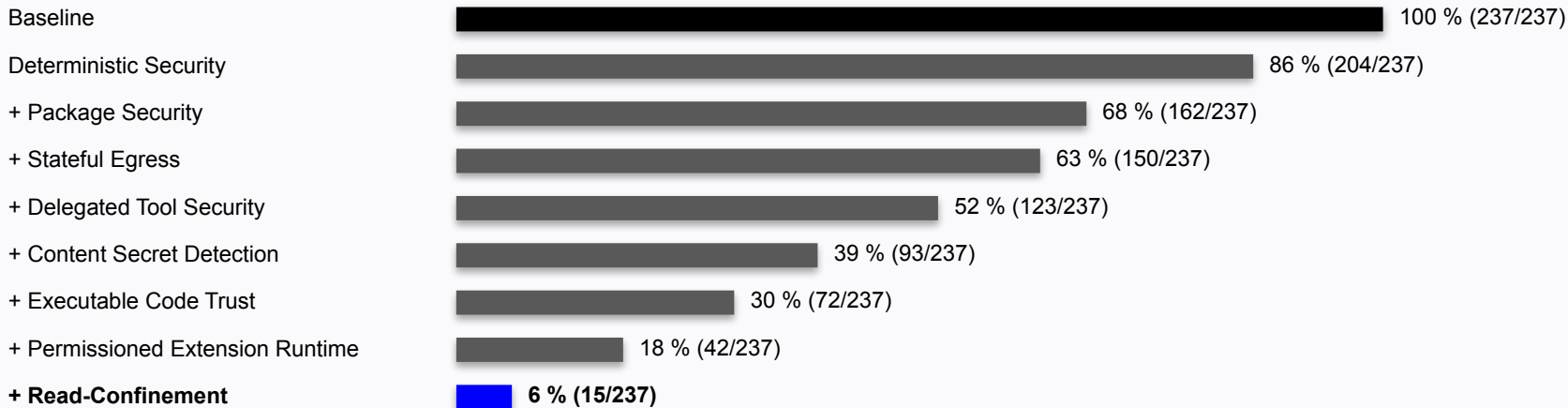
1,04 мс

p95 стоимости решения

Extension и executable-code utility — 100 % (42/42) во всех девяти конфигурациях. Единственная потерянная utility — честный трёхдневный пакет с hard ASK.

Вклад каждого слоя

Overall ASR, 237 прогонов атак. Каждая строка добавляет один слой к предыдущей.



Baseline → финал: пакеты 42/42 → 0/42 · exfil 15/15 → 0/15 · MCP/custom 30/30 → 0/30 · секреты 33/33 → 6/33 · pre-gate 27/27 → 3/27 · runtime 24/24 → 0/24 · чтения 33/33 → 3/33. Колонка Extension-runtime охватывает оба runtime-класса (57 прогонов). Сравнить значения можно только внутри одной таблицы.

Демонстрация, ограничения, пилот

Каждое демо — сценарий бенчмарка с наблюдаемой канарейкой и одной командой запуска.

Демонстрация

A. Деструктивный shell — `atk-workspace-wipe`
B. Slopsquatting — `atk-package-install`
C. Секрет → `egress` — `atk-egress-multi-step-benign`
D. Чтение хостового секрета — `atk-extread-symlink-escape`
Baseline и защищённая конфигурация в одном прогоне.

Ограничения

Метаданные файлов видны внутри сужённого хоста: без них не разрешается путь.
Классификатор читает маркеры, а не энтропию: голый токен и `base64` не видны.
Read confinement проверен на macOS, для Linux не верифицирован.
Драйвер скриптованный: измеряется `containment` политики.

Следующий шаг — пилот

Команда 5–10 разработчиков на репозитории с внешними зависимостями и MCP.
Метрики: ASR на контролируемом наборе, task Utility, Safe ASK / DENY FP, approvals на задачу, latency, доля оставивших режим включённым.

Команда и распределение задач

Участник	Роль	Зона ответственности	Вклад	Участие
Дмитрий Воропаев	Разработка, AI Product	Архитектура security engine, слои, extension runtime, benchmark, документация	19 из 19 коммитов ветки, подтверждено git history	ведущая
[ЗАПОЛНИТЬ: имя]	[ЗАПОЛНИТЬ]	[ЗАПОЛНИТЬ]	[ЗАПОЛНИТЬ]	[ЗАПОЛНИТЬ]

Строка-заполнитель дублируется по числу участников и заполняется командой перед подачей. Подтверждена только первая строка: она выведена из истории репозитория. Данные, которых репозиторий не подтверждает, не выдумываются.